

SVG

Scalable Vector Graphics

Tvorba internetových aplikací

RNDr. Richard Ostertág, PhD.

KI FMFI UK Bratislava

ostertag@dcs.fmph.uniba.sk

Sources

- Mozilla Developer Network: SVG
 - <https://developer.mozilla.org/en-US/docs/SVG/Tutorial>
- W3C – An SVG Primer for Today's Browsers
 - <http://www.w3.org/Graphics/SVG/IG/resources/svgprimer.html>
- Scalable Vector Graphics (SVG) 1.1 (2nd ed.)
 - <http://www.w3.org/TR/2011/REC-SVG11-20110816/>
- [SVG Essentials, By J. David Eisenberg](#)
 - <http://oreilly.com/catalog/svgess/chapter/ch03.html>

SVG

- Scalable Vector Graphics, is a
 - an application of XML (similar to XHTML)
 - vector graphics (including fonts, animations, ...)
 - became a W3C recommendation in September 2001

Example standalone SVG file

```
<?xml version="1.0" standalone="no"?>
```

```
<svg xmlns="http://www.w3.org/2000/svg" version="1.1">
```

```
  <circle cx="100" cy="50" r="40" stroke="black" stroke-width="2" fill="red" />
```

```
</svg>
```

SVG tools

- Inkscape – open source vector graphics editor
 - <http://inkscape.org/>
 - capabilities similar to Illustrator, CorelDraw
 - uses the W3C standard SVG file format
- high quality open clipart community
 - <http://openclipart.org/>
 - uses SVG file format
- <http://www.w3.org/QA/Tools/>
 - Unicorn – W3C's Unified Validator
 - Markup Validator – checks web documents in formats like HTML and XHTML, **SVG** or MathML

SVG – pros

- resolution independence
- Document Object Model
 - create SVG document
 - inspect SVG elements
 - programmatically manipulate shapes, styles, and positions (animation)
 - attach event handlers to SVG nodes (interaction)
- no unnecessary HTTP requests (compared to)
- supported in nearly every browser (today)
 - does not require 3rd party extensions to work
- for large images
 - faster than canvas
 - smaller than bitmap images
- true hyperlinking (instead of image maps)

SVG – cons

- for complex images (thousands of nodes)
 - do the rendering in canvas
 - use bitmap images
- if support for IE8 is required
 - you must either provide another vector fallback
 - e.g. VML
 - do not use vector at all
 - instead rely on responsively sized bitmap images

XML namespaces

Specified in parent element

```
<?xml version="1.0" standalone="no"?>
<parent
  xmlns="http://example.org"
  xmlns:svg="http://www.w3.org/2000/svg">
  <!-- parent contents here -->
  <svg:svg width="4cm" height="8cm"
    version="1.1">
    <svg:ellipse cx="2cm" cy="4cm" rx="2cm" ry="1cm" />
  </svg:svg>
  <!-- parent contents here -->
</parent>
```

Specified in <svg> element

- as the default namespace

```
<svg xmlns="http://www.w3.org/2000/svg" ...>
  <rect ... />
</svg>
```
- as an explicit namespace

```
<svg:svg xmlns:svg="http://www.w3.org/2000/svg" ...>
  <svg:rect ... />
</svg:svg>
```

SVG Authoring Guidelines 1/2

- send the correct headers with the SVG files
 - Content-Type: image/svg+xml
- for gzipped SVG files
 - Content-Type: image/svg+xml
 - Content-Encoding: gzip
- don't include a DOCTYPE declaration
 - include the “version” and “baseProfile” attributes
- bind the required namespaces
 - the SVG and XLink namespaces at a minimum
 - possibly also the XML Events namespace

```
<svg version="1.1"
      baseProfile="full"
      xmlns="http://www.w3.org/2000/svg"
      xmlns:xlink="http://www.w3.org/1999/xlink"
      xmlns:ev="http://www.w3.org/2001/xml-events">
```

SVG Authoring Guidelines 2/2

- avoid the “style” attribute where possible
 - instead of: `<circle style="fill:red; stroke:blue;" ...`
 - write: `<circle fill="red" stroke="blue" ...`
 - i.e. use presentation attributes
 - XML attribute that share the same name with their corresponding CSS property
- specify units in presentation properties
 - instead of: `<text stroke-width="2" ...>`
 - write: `<text stroke-width="2px" ...>`
- use namespace aware DOM methods
 - instead of `createElement` use `createElementNS`, etc.
 - `createElementNS('http://www.w3.org/2000/svg', 'rect');`
 - `getAttributeNS(null, 'x');` // `<rect x = ... />`
 - the namespace name for attributes without a prefix does not have a value (i.e. = null)
 - `getAttributeNS('http://www.w3.org/1999/xlink', 'href');` // `<a xlink:href = ... >`
 - the rule is simple:
 - **for elements** with or without a namespace prefix, and **for attributes with a namespace prefix use the namespace URI** of the element/attribute in question
 - **for attributes without** a namespace prefix the **namespace name is null**

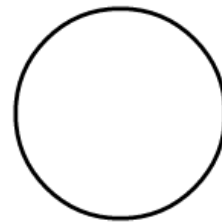
The coordinating system

- the SVG canvas is infinite
- viewport – the area of the canvas that your document intends to use
 - size of viewport is defined by width and height attributes of the <svg> element
 - em – The font size of the default font
 - ex – The height of the letter x
 - px – Pixels (default if units are not given)
 - pt – Points (1/72 of an inch)
 - pc – Picas (1/6 of an inch)
 - cm – Centimeters
 - mm – Millimeters
 - in – Inches
- specifying units in the <svg> element does not affect coordinates given without units in other elements if no viewBox attr. is used
 - you can redefine what the coordinates without units mean inside an <svg> element using the viewBox attribute
- viewBox="minX minY width height"
 - then (minX px,minY px) is at top left corner
 - 1px horizontally is equal to SVG width / viewBox width
 - 1px vertically is equal to SVG height / viewBox height
- X coordinate increases to the right
- Y coordinate increases to the bottom

SVG in SVG

- add another <svg> element for the new viewport
- the x and y attributes specifies where the new viewport is to be established
 - in terms of the enclosing <svg> element
 - if omitted they defaults to zero

```
<svg width="200px" height="200px" viewBox="0 0 200 200">  
  <circle cx="25" cy="25" r="25" stroke="black" fill="none" />  
  <rect x="100" y="5" width="30" height="80" stroke="blue" fill="none" />  
  <svg x="100px" y="5px" width="30px" height="80px" viewBox="0 0 30 80">  
    <circle cx="15" cy="40" r="30" stroke="black" fill="none" />  
  </svg>  
</svg>
```



How to specify the SVG colors

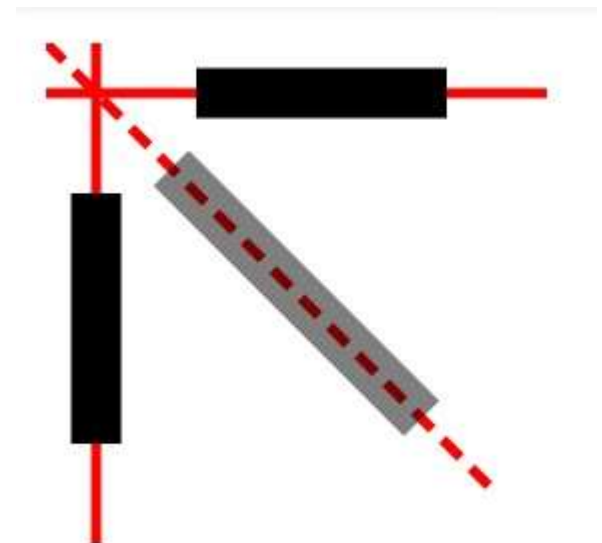
- by one of the color keyword names:
 - "aqua", "black", "blue", "fuchsia", "gray", "green", "lime", "maroon", "navy", "olive", "purple", "red", "silver", "teal", "white" and "yellow"
- by a six-digit hexadecimal specifier in the form #rrggbb
- by a three-digit hexadecimal specifier in the form #rgb
 - equivalent to #rrggbb
- by an rgb specifier in the form
 - rgb(red-value, green-value, blue-value)
 - each value is
 - from 0 to 255
 - a percentage from 0% to 100%
- By an rgba specifier in the form
 - rgba(red-value, green-value, blue-value, alpha-value)
 - each RGB value is from 0 to 255
 - alpha value is from 0 to 1
 - combines with opacity attrib.

SVG <line> element

```
<svg width="200px" height="200px" viewBox="0 0 200 200">  
  <!-- horizontal line -->  
  <line x1="0" y1="10" x2="100" y2="10" stroke-width="2px" stroke="red"/>  
  <line x1="30" y1="10" x2="80" y2="10" stroke-width="10px" stroke="black"/>  
  <!-- vertical line -->  
  <line x1="10" y1="0" x2="10" y2="100" stroke-width="2px" stroke="red"/>  
  <line x1="10" y1="30" x2="10" y2="80" stroke-width="10px" stroke="black"/>  
  <!-- diagonal line -->  
  <line x1="0" y1="0" x2="90" y2="90" stroke-width="2px" stroke="red" stroke-dasharray="5px 3px"/>  
  <line x1="25" y1="25" x2="75" y2="75" stroke-width="10px" stroke="black" stroke-opacity="0.5"/>
```

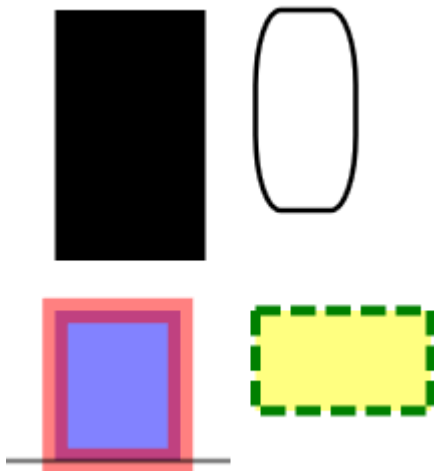
</svg>

- <line x1="start-x" y1="start-y" x2="end-x" y2="end-y">
 - stroke-width
 - stroke (color)
 - stroke-opacity
 - stroke-dasharray



Basic shapes – rectangle

- `<rect x="10px" y="10px" width="30px" height="50px"/>`
- the rectangle's sides in the current user coordinate system are defined by
 - the x: is the smaller of the x-coordinate values
 - the y: is the smaller of the y-coordinate values
 - without transformations this is equivalent to the upper left corner



```
<svg width="500px" height="500px" viewBox="0 0 200 200">  
  <!-- black interior, no outline -->  
  <rect x="10px" y="10px" width="30px" height="50px"/>
```

```
<!-- no interior, black outline, rounded corners -->  
<rect x="50px" y="10px" width="20px" height="40px"  
  rx="5px" ry="15px" fill="none" stroke="black"/>
```

```
<!-- semi-transparent blue interior, thick semi-transparent red  
outline -->  
<rect x="10px" y="70px" width="25px" height="30px"  
  fill="rgba(0,0,255,0.7)" fill-opacity="0.7" stroke="red"  
  stroke-width="5px" stroke-opacity="0.5"/>
```

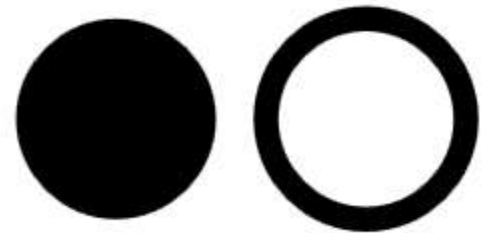
```
<line x1="0px" y1="100px" x2="45px" y2="100px"  
  stroke="black" stroke-width="1px" stroke-opacity="0.5"/>
```

```
<!-- semi-transparent yellow interior, dashed green outline -->  
<rect x="50px" y="70px" width="35px" height="20px"  
  fill="yellow" fill-opacity="0.5" stroke="green" stroke-  
  width="2px" stroke-dasharray="5px 2px"/>  
</svg>
```

Basic shapes – circle

- `<circle cx="30px" cy="30px" r="20px" />`
 - (cx, cy) is the center (if omitted, it is presumed to be zero)
 - r is radius (if zero, no shape is displayed, negative values are invalid)
 - the default is black fill and no outline

```
<svg width="500px" height="500px" viewBox="0 0 200 200">  
  <circle cx="30px" cy="30px" r="20px" />  
  <circle cx="80px" cy="30px" r="20px"  
    stroke-width="5px" stroke="black" fill="none"/>  
</svg>
```



Basic shapes – ellipse

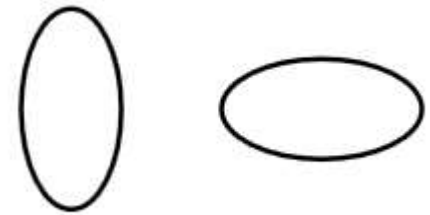
- `<ellipse cx="30px" cy="80px" rx="10px" ry="20px" />`
 - (cx, cy) is the center (if omitted, it is presumed to be zero)
 - rx, ry is x-radius and y-radius (if zero, no shape is displayed, negative values are invalid)
 - the default is black fill and no outline

```
<svg width="500px" height="500px" viewBox="0 0 200 200">
```

```
  <ellipse cx="30px" cy="80px" rx="10px" ry="20px" stroke="black" fill="none" />
```

```
  <ellipse cx="80px" cy="80px" rx="20px" ry="10px" stroke="black" fill="none" />
```

```
</svg>
```



Basic shapes – polygon

- `<polygon points="x1,y1 x2,y2 ..." />`
 - don't have to return to the starting point, the shape will automatically be closed

```
<svg width="500px" height="500px" viewBox="0 0 200 200">
```

```
<polygon
```

```
  points="35,37.5 37.9,46.1 46.9,46.1 39.7,51.5 42.3,60.1 35,55 27.7,60.1
```

```
          30.3,51.5 23.1,46.1 32.1,46.1"
```

```
  fill="green" fill-opacity="0.25" stroke="green"/>
```

```
</svg>
```



Basic shapes – polygon fill-rule

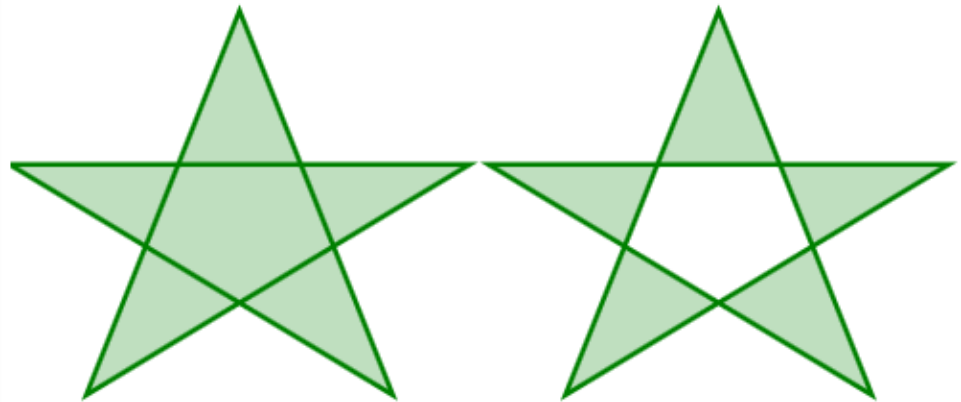
- fill-rule
 - nonzero (default)
 - evenodd

```
<svg width="500px" height="500px"  
      viewBox="0 0 200 200">
```

```
<polygon  
  fill-rule="nonzero"  
  points="48,16 16,96 96,48 0,48 80,96"  
  fill="green" fill-opacity="0.25" stroke="green"/>
```

```
<polygon  
  fill-rule="evenodd"  
  points="148,16 116,96 196,48 100,48 180,96"  
  fill="green" fill-opacity="0.25" stroke="green"/>
```

```
</svg>
```



Basic shapes – polyline

- use for a series of lines that does not make a closed shape
 - specify fill="none"
 - otherwise, the SVG viewer attempts to fill the shape
 - but closing line will be still missing (only diff. to polygon)

```
<svg width="500px" height="500px" viewBox="0 0 200 200">
```

```
<polyline
```

```
  points="5 20, 20 20, 25 10, 35 30, 45 10, 55 30, 65 10, 75 30, 80 20, 95 20"
```

```
  stroke="black" stroke-width="3px" fill="none" />
```

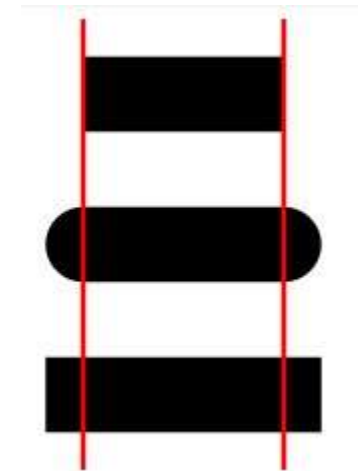
```
</svg>
```



Line Caps

- specify the shape of the endpoints of the lines
- attribute: `stroke-linecap`
 - values `"butt"`, `"round"` or `"square"`

```
<svg width="500px" height="500px" viewBox="0 0 200 200">  
  <line x1="10px" y1="15px" x2="50px" y2="15px"  
    stroke="black" stroke-width="15px" stroke-linecap="butt"/>>  
  <line x1="10px" y1="45px" x2="50px" y2="45px"  
    stroke="black" stroke-width="15px" stroke-linecap="round"/>>  
  <line x1="10px" y1="75px" x2="50px" y2="75px"  
    stroke="black" stroke-width="15px" stroke-linecap="square">  
  <line x1="10px" y1="0px" x2="10px" y2="90px" stroke="red"/>  
  <line x1="50px" y1="0px" x2="50px" y2="90px" stroke="red"/>  
</svg>
```



Line Joins

- specify the way lines connect at the corners
- stroke-linejoin
 - values "miter", "round" or "bevel"
- stroke-miterlimit
 - max. ratio of length of the miter point to the width of the lines being drawn
 - the default value is 4

```
<svg width="500px" height="500px"
      viewBox="0 0 200 200">
  <polyline points="30 30, 45 15, 60 30"
    stroke="black" stroke-width="15"
    fill="none" stroke-linejoin="miter" />
  <polyline points="90 30, 105 15, 120 30"
    stroke="black" stroke-width="15"
    fill="none" stroke-linejoin="round" />
  <polyline points="150,30 165,15 180,30"
    stroke="black" stroke-width="15"
    fill="none" stroke-linejoin="bevel" />
</svg>
```



The <image> element

- includes an entire SVG or raster file
 - if you are including an SVG file, the x, y, width, and height attributes establish the viewport in which the referenced file will be drawn
 - if you're including a raster file, it will be scaled to fit the rectangle that the attributes specify
 - recognize JPEG or PNG raster files

```
<svg width="310px" height="310px" viewBox="0 0 310 310">  
  <ellipse cx="154" cy="154" rx="100" ry="100" fill="#999" />  
  <ellipse cx="152" cy="152" rx="100" ry="100" fill="#cef" />  
  <image xlink:href="http://.../logoFMFIttransp.png"  
    x="72" y="92" width="160" height="120" />  
</svg>
```



Grouping – the “g” element

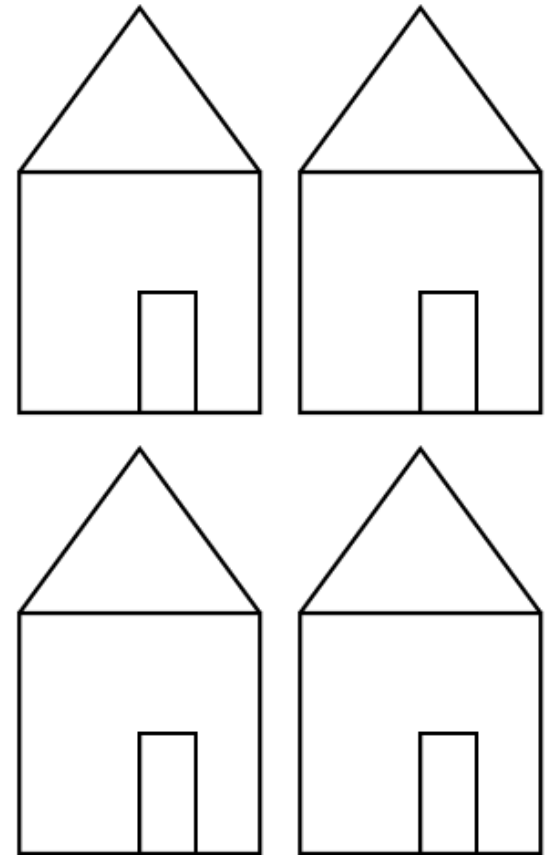
- The ‘g’ element is a container element for grouping together related graphics elements.
- It often has an id attribute to give that group a unique name.
- each group may have its own <title> and <desc>
 - to identify it for text-based XML applications
 - to aid in accessibility for visually-impaired users
 - browsers may displays title as tooltip

```
<svg width="240px" height="240px"
      viewBox="0 0 240 240">
  <title>Grouped Drawing</title>
  <desc>Stick-figure drawings of a house</desc>
  <g id="house" style="fill: none; stroke: black;">
    <desc>House with door</desc>
    <rect x="6" y="50" width="60" height="60" />
    <polyline points="6 50, 36 9, 66 50" />
    <polyline points="36 110, 36 80, 50 80, 50 110" />
  </g>
</svg>
```

The <use> element

- display element (even group) again with the <use> tag
 - xlink:href attribute specifies URI of used element
 - specify the x and y location (relative to original)
 - the fill and stroke color are established by the original
 - references only to objects already created

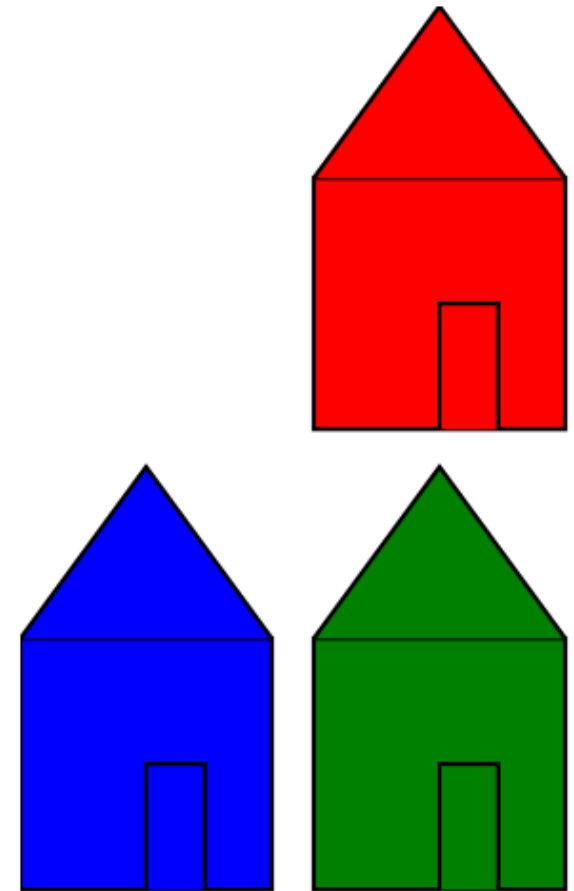
```
<svg width="400px" height="480px" viewBox="0 0 200 240">  
  <title>Grouped Drawing</title>  
  <desc>Stick-figure drawings of houses</desc>  
  <g id="house" style="fill: none; stroke: black;">  
    <desc>House with door</desc>  
    <rect x="6" y="50" width="60" height="60" />  
    <polyline points="6 50, 36 9, 66 50" />  
    <polyline points="36 110, 36 80, 50 80, 50 110" />  
  </g>  
  <use xlink:href="#house" x="70" y="0"/>  
  <use xlink:href="#house" x="0" y="110"/>  
  <use xlink:href="#house" x="70" y="110"/>  
</svg>
```



The <defs> element

- elements in defs are defined without displaying
- the SVG specification recommends to put all objects for re-use within a <defs> element so that SVG viewers working in a streaming environment can process data more efficiently

```
<svg width="400px" height="480px" viewBox="0 0 200 240">
  <title>Grouped Drawing</title>
  <desc>Stick-figure drawings of a house</desc>
  <defs>
    <g id="house" style="stroke: black;">
      <desc>House with door</desc>
      <rect x="0" y="41" width="60" height="60" />
      <polyline points="0 41, 30 0, 60 41" />
      <polyline points="30 101, 30 71, 44 71, 44 101" />
    </g>
  </defs>
  <use xlink:href="#house" x="70" y="0" fill="red" />
  <use xlink:href="#house" x="0" y="110" fill="blue" />
  <use xlink:href="#house" x="70" y="110" fill="green" />
</svg>
```



Stylesheets

Internal

```
<svg width="200px" height="200px" viewBox="0 0 200 200">
  <defs>
    <style type="text/css"><![CDATA[
      * {fill:none; stroke: black}
      rect {stroke-dasharray: 7 3}
      circle.yellow {fill: yellow}
      .thick {stroke-width: 5}
      .semiblue {fill:blue; fill-opacity: 0.5}
    ]]></style>
  </defs>
  <line x1="10" y1="10" x2="40" y2="10" />
  <rect x="10" y="20" width="40" height="30" />
  <circle class="yellow" cx="70" cy="20" r="10" />
  <polygon class="thick" points="60 50, 60 80, 90 80" />
  <polygon class="thick semiblue"
    points="100 30, 150 30, 150 50, 130 50" />
</svg>
```

External

```
<?xml version="1.0"?>
<?xml-stylesheet href="ext_style.css" type="text/css"?>
<svg width="200px" height="200px" viewBox="0 0 200 200">
  <line x1="10" y1="10" x2="40" y2="10" />
  <rect x="10" y="20" width="40" height="30" />
  <circle class="yellow" cx="70" cy="20" r="10" />
  <polygon class="thick" points="60 50, 60 80, 90 80" />
  <polygon class="thick semiblue"
    points="100 30, 150 30, 150 50, 130 50" />
</svg>
```

ext_style.css:

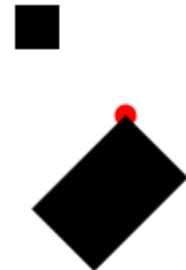
```
* {fill:none; stroke: black}
rect {stroke-dasharray: 7 3}
circle.yellow {fill: yellow}
.thick {stroke-width: 5}
.semiblue {fill:blue; fill-opacity: 0.5}
```

- any style specification (inline, internal or external) override a presentation attribute

Transforming the Coordinate System

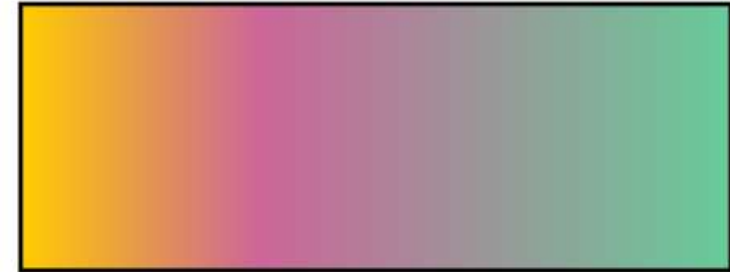
- transform attribute
 - translate(dx, dy), rotate(angle), scale(s), scale(sX,sY)
 - skewX(angle), skewY(angle)
 - angle is in degrees

```
<svg width="200px" height="200px" viewBox="0 0 200 200">  
  <g id="square">  
    <rect x="0" y="0" width="20" height="20"  
      style="fill: black; stroke-width: 2;" />  
  </g>  
  <circle cx="50" cy="50" r="5" fill="red"/>  
  <use xlink:href="#square"  
    transform="translate(50,50) rotate(45) scale(2,3)" />  
</svg>
```



Gradients – linear

```
<svg width="200px" height="200px" viewBox="0 0 200 200">
  <defs>
    <linearGradient id="three_stops">
      <stop offset="0%" style="stop-color: #ffcc00;" />
      <stop offset="33.3%" style="stop-color: #cc6699" />
      <stop offset="100%" style="stop-color: #66cc99;" />
    </linearGradient>
    <linearGradient id="diagonal"
      xlink:href="#three_stops"
      x1="0%" y1="0%" x2="100%" y2="100%" />
  </defs>
  <rect x="20" y="20" width="160" height="60"
    style="fill: url(#three_stops); stroke: black;" />
  <rect x="20" y="120" width="160" height="60"
    style="fill: url(#diagonal); stroke: black;" />
</svg>
```



Text

```
<svg width="500px" height="500px" viewBox="0 0 200 200">  
  <path d="M 20 10, 20 120 M 10 30 100 30 M 10 70 100 70  
    M 10 110 100 110" style="stroke: red;" />  
  <text x="20" y="30">Simplest Text</text>  
  <text x="20" y="70" style="stroke: black; stroke-width: 0.5;">  
    Outlined/filled</text>  
  <text x="20" y="110" style="stroke: black;  
    stroke-width: 0.5; fill: none;">Outlined only</text>  
</svg>
```

Simplest Text

Outlined/filled

Outlined only

Text on the path

```
<svg width="500" height="500" viewBox="0 0 400 400">
  <defs>
    <path id="curvpath" d="M30 40 C 50 10, 70 10, 120 40 S 150 0, 200 40" style="stroke: gray; fill: none;" />
    <path id="round-corner" d="M250 30 L 300 30 A 30 30 0 0 1 330 60 L 330 110" style="stroke: gray; fill: none;" />
    <path id="sharp-corner" d="M 30 110 100 110 100 160" style="stroke: gray; fill: none;" />
    <path id="discontinuous" d="M 150 110 A 40 30 0 1 0 230 110 M 250 110 270 140" style="stroke: gray; fill: none;" />
  </defs>
  <text style="font-size: 12;">
    <textPath xlink:href="#curvpath">Following a cubic Bézier curve.</textPath>
  </text>
  <text style="font-size: 12;">
    <textPath xlink:href="#round-corner">Going 'round the bend</textPath>
  </text>
  <text style="font-size: 12;">
    <textPath xlink:href="#sharp-corner">
      Making a quick turn</textPath>
  </text>
  <text style="font-size: 12;">
    <textPath xlink:href="#discontinuous">
      Text along a broken path</textPath>
  </text>
</svg>
```

